

Practical - 5

Aim: Implementation of a knapsack problem using dynamic programming.

Algorithm :

Knapsack Algorithm

Input : n : Total number of items
 W : maximum capacity of the knapsack

Output : Maximum value/profit that can be accommodated in the knapsack of capacity W .

Algorithm :

- ① Start
- ② create a 2D array k of size $(n+1) \times (W+1)$ initialized to 0.
- ③ Iterate through items and capacities :
 - Loop i from 0 to n (rows representing items)
 - Loop w from 0 to W (columns representing weights) -
- ④ Base case :
 - If $i=0$ or $w=0$, set $k[i][w]=0$
- ⑤ Fill the DP Table :
 - For each item i and capacity w :
 - If the item weight fits ($wt[i-1] \leq w$) :
$$k[i][w] = \max(val[i-1] + k[i-1][w - wt[i-1]])$$
 - If the item exceeds capacity ($wt[i-1] > w$) :
$$k[i][w] = k[i-1][w]$$
- ⑥ Retrieve Result : The maximum profit is stored at $k[n][W]$.

Program:

```
#include <iostream>
#include <algorithm>
using namespace std;

class Knapsack {
private:
    int w[10];
    int v[10];
    int n;
    int W;
    int dp[10][100];

public:
    void getInput() {
        cout << "Enter number of items: ";
        cin >> n;
        cout << "Enter knapsack capacity: ";

        cin >> W;

        for (int i = 1; i <= n; i++) {
            cout << "Item " << i << " weight and value: ";
            cin >> w[i] >> v[i];
        }
    }

    void solve() {
        for (int i = 0; i <= n; i++)
            dp[i][0] = 0;

        for (int j = 0; j <= W; j++)
            dp[0][j] = 0;

        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= W; j++) {
                if (w[i] <= j) {
                    dp[i][j] = max(
                        v[i] + dp[i - 1][j - w[i]],
                        dp[i - 1][j]
                    );
                } else {
                    dp[i][j] = dp[i - 1][j];
                }
            }
        }
    }
}
```

Name: Vemalatha Bhimireddy
Enrollment Number:92460118162



```
void printTable() {
    cout << "\nDP Table:\n";

    cout << " ";
    for (int j = 0; j <= W; j++)
        cout << j << " ";
    cout << endl;

    for (int i = 0; i <= n; i++) {
        cout << "i=" << i << " ";
        for (int j = 0; j <= W; j++)
            cout << dp[i][j] << " ";
        cout << endl;
    }
}

void findItems() {
    cout << "\nSelected Items:\n";

    int j = W;

    for (int i = n; i >= 1; i--) {
        if (dp[i][j] != dp[i - 1][j]) {
            cout << "Item " << i
                << " (w=" << w[i]
                << ", v=" << v[i]
                << ") TAKEN\n";
            j -= w[i];
        } else {
            cout << "Item " << i
                << " (w=" << w[i]
                << ", v=" << v[i]
                << ") NOT taken\n";
        }
    }
}

void showResult() {
    cout << "\nMaximum Value = " << dp[n][W] << endl;
}

};

int main() {
    Knapsack k;

    k.getInput();
    k.solve();
    k.printTable();
    k.showResult();
}
```

```
k.findItems();  
  
return 0;  
}
```

Output :

```
PS C:\Users\vemal\Downloads\data structure> g++ knapsack.cpp -o knapsack  
PS C:\Users\vemal\Downloads\data structure> .\knapsack  
Enter number of items: 5  
Enter knapsack capacity: 4  
Item 1 weight and value: 7  
7  
Item 2 weight and value: 4 8  
Item 3 weight and value: 3 6  
Item 4 weight and value: 5 7  
Item 5 weight and value: 7 9  
  
DP Table:  
  0 1 2 3 4  
i=0 0 0 0 0 0  
i=1 0 0 0 0 0  
i=2 0 0 0 0 8  
i=3 0 0 0 6 8  
i=4 0 0 0 6 8  
i=5 0 0 0 6 8  
  
Maximum Value = 8  
  
Selected Items:  
Item 5 (w=7, v=9) NOT taken  
Item 4 (w=5, v=7) NOT taken  
Item 3 (w=3, v=6) NOT taken  
Item 2 (w=4, v=8) TAKEN  
Item 1 (w=7, v=7) NOT taken
```

Time complexity :

```
Time complexity :
for (int i = 1; i ≤ n; i++) { ——— n
    for (int j = 1; j ≤ W; j++) { ——— W
        if (w[i] ≤ j) { ——— 1
            dp[i][j] = max ( ——— 1
                v[i] + dp[i-1][j-w[i]], ——— 1
                dp[i-1][j] ——— 1
            );
        }
    }
}
```

$T(n) = n \times W$
 $T(n) = O(nW)$

Best case : The nested loop still runs every item and every capacity, even if many items cannot fit.
 $T(n) = O(nW)$

Average case : On average, the program still fills the complete DP table of $n \times W$.
 $T(n) = O(nW)$

Worst case : The outer loop runs n -times and the inner loop runs W -times for each item.
 $T(n) = O(nW)$

* For this knapsack program, best case, Average case and worst case are all $O(nW)$. This is because the program uses two nested loops.

Conclusion : The Knapsack problem was successfully implemented using dynamic programming. This method efficiently finds the maximum value that can be obtained within the given knapsack capacity while avoiding unnecessary repeated calculations.



Marwadi
University
Marwadi Chandarana Group



Faculty of Engineering & Technology
Department of CSE-AI&ML
Design and analysis of algorithm- 01AI0506